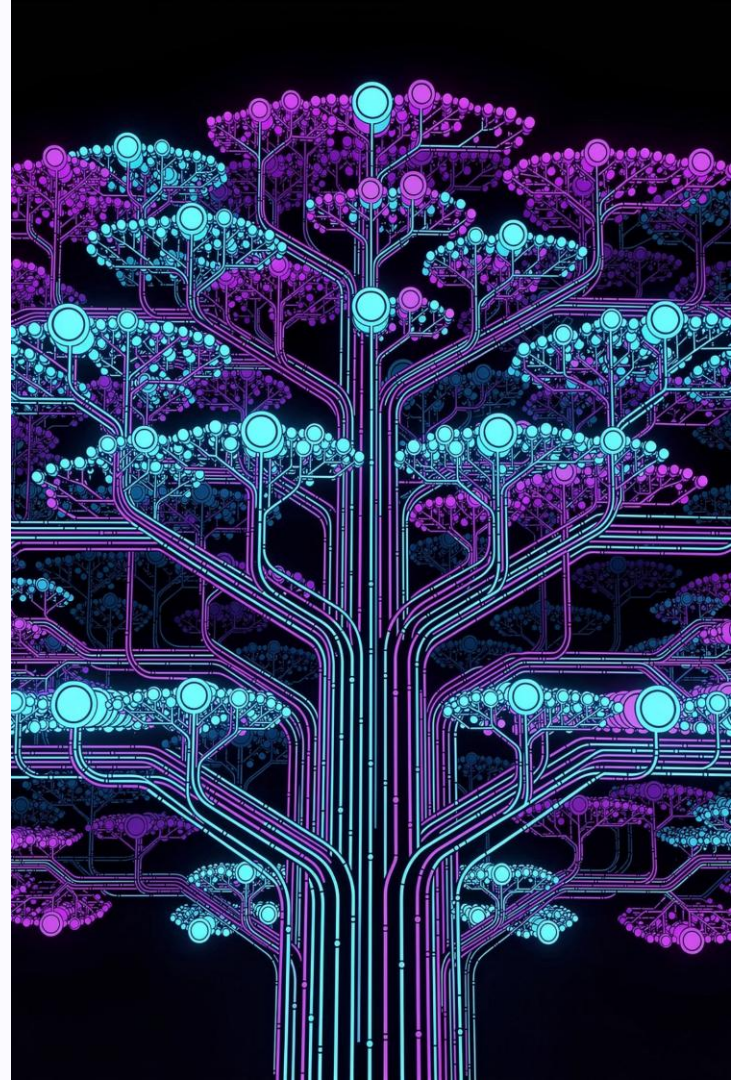


Análisis de Complejidad de Algoritmos Recursivos

Introducción a la Computación II — Ingeniería en Sistemas



¿Por qué importa la complejidad algorítmica?

Un algoritmo puede ser **correcto** y aun así ser **inutilizable** en la práctica. La diferencia entre un buscador que responde en milisegundos y uno que tarda minutos no es magia: es la elección del algoritmo correcto.



Inteligencia Artificial

Los modelos de aprendizaje profundo deben procesar millones de parámetros; elegir algoritmos eficientes reduce horas de entrenamiento a minutos.



Videojuegos

Los motores de búsqueda de rutas como A* deben ejecutarse en tiempo real para NPCs. Un algoritmo $O(n^2)$ arruinaría la fluidez del juego.



Motores de búsqueda

Google indexa billones de páginas. Sin algoritmos de complejidad logarítmica, cada búsqueda tardaría años en completarse.

  **Dato curioso:** Un algoritmo $O(2^n)$ con $n=60$ requeriría más de 1.000 años en la computadora más rápida del mundo.

¿Qué es la Recursividad?

Una función es **recursiva** cuando se llama a sí misma para resolver una versión más pequeña del mismo problema. Como las muñecas rusas: cada una contiene una versión más pequeña hasta llegar a la última.

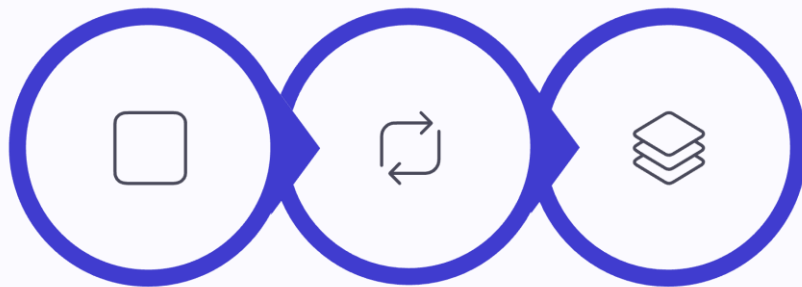
Todo algoritmo recursivo tiene:

- **Caso base:** la condición que detiene la recursión (sin esto → bucle infinito).
- **Llamada recursiva:** la función se invoca con un subproblema más pequeño.
- **Progresión:** cada llamada acerca el problema al caso base.

Pseudocódigo general

```
función resolver(n):  
    si n == caso_base:  
        retornar solución_directa  
    sino:  
        subresultado = resolver(n-1)  
        retornar combinar(subresultado)
```

🤔 **Reflexión:** ¿Qué ocurre si olvidamos definir el caso base?



Caso Base

Llamada
Recursiva

Resultado
Combinado

Ejemplo Básico: Factorial Recursivo

El factorial de n ($n!$) es el producto de todos los enteros desde 1 hasta n . Es el ejemplo canónico de recursividad: $n! = n \times (n-1)!$

Código en C++

```
#include <iostream>
using namespace std;

int factorial(int n) {
    // Caso base
    if (n == 0 || n == 1) {
        return 1;
    }
    // Llamada recursiva
    return n * factorial(n - 1);
}

int main() {
    int numero;
    cout << "Ingrese un número: ";
    cin >> numero;

    cout << "El factorial de " << numero << " es: " << factorial(numero) << endl;

    return 0;
}
```

Expansión de factorial(4)

factorial(4) = $4 \times \text{factorial}(3)$

factorial(3) = $3 \times \text{factorial}(2)$

factorial(2) = $2 \times \text{factorial}(1)$

factorial(1) = $1 \leftarrow$ Caso Base ✓

✓ Cada llamada reduce n en 1, garantizando que siempre se alcanza el caso base. El retorno construye la solución de abajo hacia arriba: $1 \rightarrow 2 \rightarrow 6 \rightarrow \mathbf{24}$

Coste Computacional de la Recursión

La recursión tiene un coste **doble**: tiempo de ejecución y uso de memoria. Cada llamada activa ocupa espacio en la **pila de llamadas (Call Stack)**, una estructura LIFO gestionada por el sistema operativo.

⌚ Tiempo de Ejecución

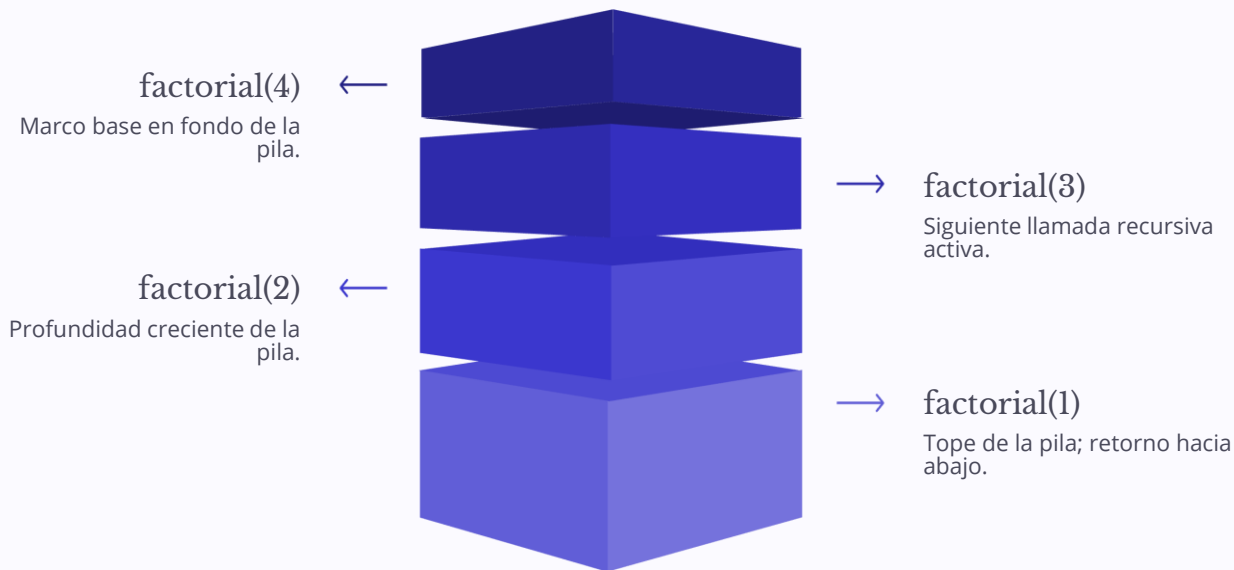
Depende del **número total de llamadas recursivas** realizadas. Cada llamada implica operaciones de entrada/salida al stack de ejecución.

🧠 Uso de Memoria

Cada llamada activa reserva un **marco de pila (stack frame)** con variables locales, parámetros y dirección de retorno.

📦 Pila de Llamadas

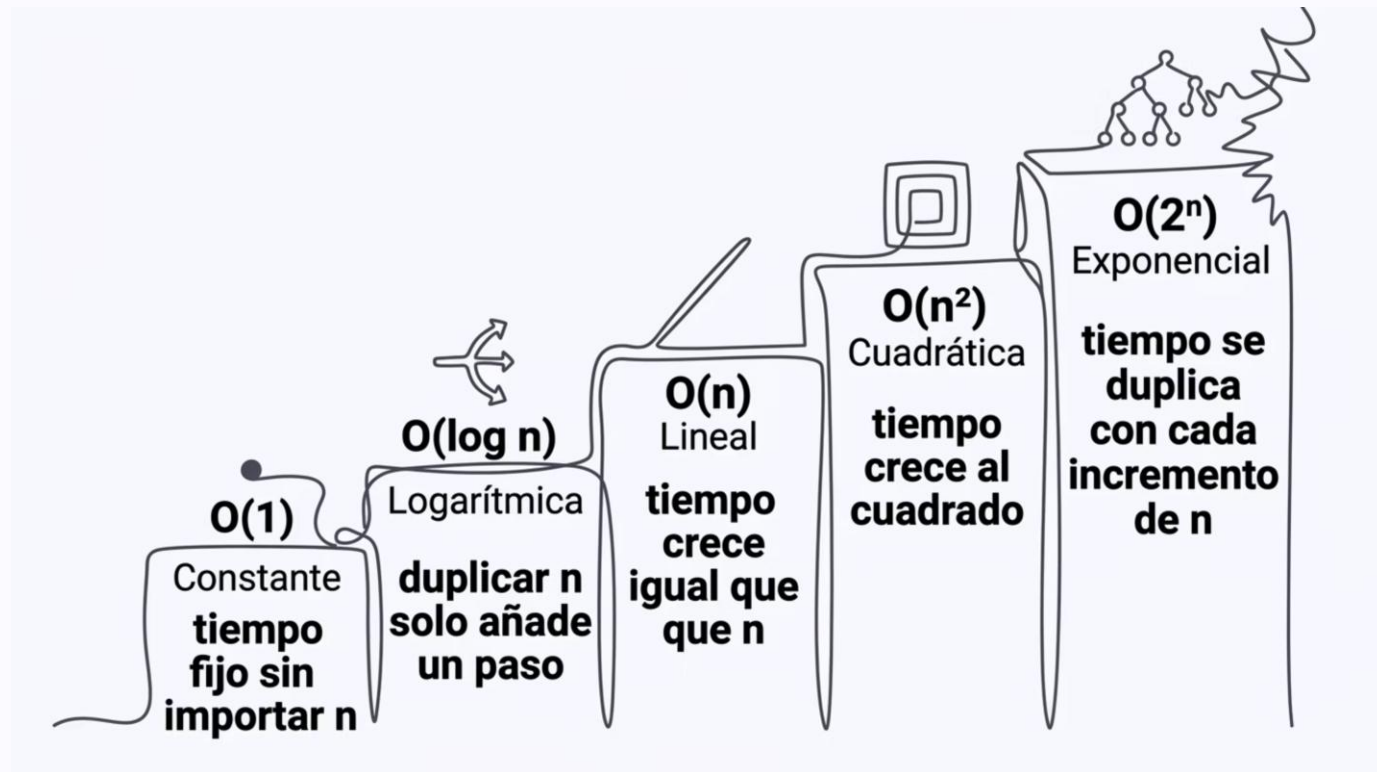
La profundidad máxima de la recursión determina el uso de memoria. Si se supera el límite → **Stack Overflow**.



En Python, el límite por defecto de la pila recursiva es de ~1000 llamadas. Para n grande, usa iteración o `sys.setrecursionlimit()`.

Notación Big O: Intuición Visual

La notación **Big O** describe cómo crece el tiempo de ejecución de un algoritmo en función del tamaño de la entrada n . No mide segundos: mide **cómo escala**.



Regla de oro: Cuanto más a la izquierda en la escalera, más eficiente es el algoritmo.

¿Cómo Analizar la Complejidad de un Algoritmo Recursivo?

El análisis de complejidad en algoritmos recursivos sigue un proceso sistemático. No es intuición: es **contar y modelar matemáticamente** el comportamiento del algoritmo.

1

Identificar llamadas recursivas

¿Cuántas veces se llama la función en cada ejecución? ¿Con qué parámetros?

2

Determinar el tamaño del subproblema

¿El problema se reduce en $n-1$, $n/2$, n/k ? Esto define la forma de la recurrencia.

3

Calcular el trabajo en cada nivel

Operaciones realizadas **fuera** de la llamada recursiva (comparaciones, sumas, asignaciones).

4

Sumar el trabajo total

Agregar el coste de todos los niveles del árbol de recursión para obtener la complejidad final.

🤔 **Reflexión:** Si una función recursiva se llama a sí misma 2 veces con $n/2$, ¿cuántas llamadas totales habría para $n=8$?

Árboles de Recursión

Un **árbol de recursión** es la herramienta visual más poderosa para analizar algoritmos recursivos. Cada nodo representa una llamada; los hijos son las llamadas que genera; las hojas son los casos base.

Cómo construir el árbol

01

Dibuja la llamada inicial como raíz

02

Expande cada llamada recursiva como hijo

03

Anota el trabajo realizado en cada nodo

04

Suma el trabajo por niveles

05

Identifica el total acumulado

Información que extrae el árbol



Altura del árbol

= profundidad de recursión =
memoria usada



Número de hojas

= cantidad de casos base
alcanzados



Trabajo por nivel

= nodos $\times$ costo por nodo

Recurrencias: El Lenguaje Matemático de la Recursión

Una **recurrencia** es una ecuación que expresa el tiempo de ejecución $T(n)$ en función del tiempo de subproblemas más pequeños. Es la traducción matemática directa del código recursivo.

Forma General

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

Donde **a** = número de llamadas, **b** = factor de reducción, **f(n)** = trabajo extra.

Factorial

$$T(n) = T(n - 1) + O(1)$$

Una llamada, problema se reduce en 1, trabajo constante → **$T(n) = O(n)$**

Búsqueda Binaria

$$T(n) = T\left(\frac{n}{2}\right) + O(1)$$

Una llamada, problema se divide a la mitad → **$T(n) = O(\log n)$**

  Las recurrencias se resuelven con el **Método Maestro**, sustitución iterativa o árboles de recursión. En este curso usaremos los árboles como herramienta principal.

Ejemplo 1: Búsqueda Binaria

La búsqueda binaria es el algoritmo recursivo más elegante: en cada paso **descarta la mitad** del espacio de búsqueda. Para $n=16$ elementos, solo necesita 4 comparaciones.

Código en C++

```
#include <iostream>
#include <vector>
using namespace std;

int binaria(const vector<int>& arr, int obj, int izq, int der) {
    if (izq > der) {
        return -1; // No encontrado
    }
    int medio = (izq + der) / 2;
    if (arr[medio] == obj) {
        return medio;
    } else if (arr[medio] < obj) {
        return binaria(arr, obj, medio + 1, der);
    } else {
        return binaria(arr, obj, izq, medio - 1);
    }
}

int main() {
    vector<int> arr = {1, 3, 5, 7, 9, 11, 13};
    int obj;
    cout << "Ingrese el número a buscar: ";
    cin >> obj;

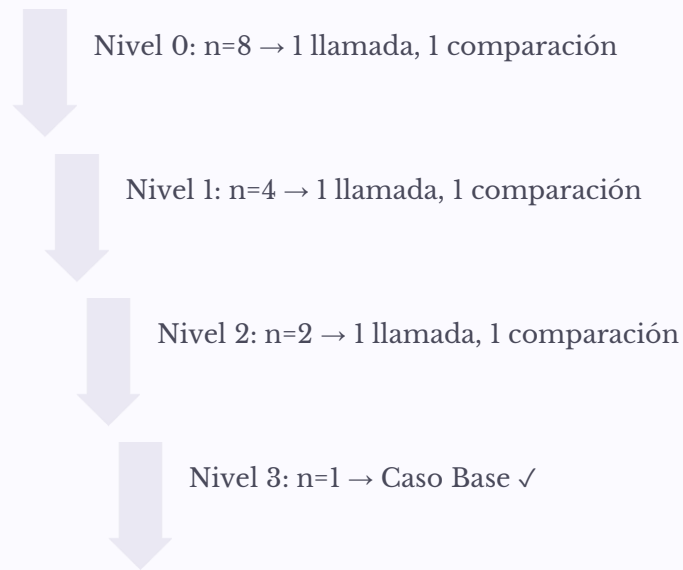
    int resultado = binaria(arr, obj, 0, arr.size() - 1);

    if (resultado != -1) {
        cout << "Elemento encontrado en la posición: " << resultado << endl;
    } else {
        cout << "Elemento no encontrado." << endl;
    }

    return 0;
}
```

Recurrencia:

Árbol de recursión ($n=8$)



✓ Altura = $\log_2(n)$ niveles $\rightarrow$ **Complejidad $O(\log n)$**
Memoria $O(\log n)$ por la pila de llamadas.

Ejemplo 2: Fibonacci Recursivo

Fibonacci parece simple pero esconde un problema grave: **recalcula los mismos valores una y otra vez**. El árbol de recursión revela la tragedia computacional.

```
#include <iostream>
using namespace std;

int fib(int n) {
    if (n <= 1) {
        return n; // Caso base
    }
    return fib(n - 1) + fib(n - 2);
}

int main() {
    int numero;
    cout << "Ingrese un número: ";
    cin >> numero;

    cout << "El término " << numero << " de la serie Fibonacci es: " << fib(numero) << endl;

    return 0;
}
```

Recurrencia:

$$T(n) = T(n - 1) + T(n - 2) + O(1)$$

⚠️ ⚠️ Llamadas repetidas: fib(3) se calcula **3 veces** para fib(5).
Para fib(50), son más de 2^{25} llamadas redundantes.

Árbol de fib(5) — llamadas repetidas

fib(5) → 2 ramas

fib(4) + fib(3)

fib(4) → 2 ramas

fib(3) + fib(2) ← repetido

Total: 15 llamadas

para n=5. Para n=50: billones.



Complejidad: $O(2^n)$ — exponencial. Completamente inviable para $n > 40$.

Fibonacci: Recursivo vs. Dinámico

La **Programación Dinámica** (memoización) elimina los cálculos redundantes almacenando resultados ya computados. El mismo problema, una solución radicalmente más eficiente.

Característica	🌲 Recursivo Simple	🚀 Dinámico (Memo)
Complejidad Temporal	$O(2^n)$ — Exponencial	$O(n)$ — Lineal
Complejidad Espacial	$O(n)$ — pila	$O(n)$ — caché + pila
fib(10)	177 llamadas	10 llamadas
fib(40)	~331 M llamadas	40 llamadas
fib(100)	Imposible (años)	Instantáneo
Llamadas repetidas	Sí — muchas	No — cada valor 1 vez

```
#include <iostream>
#include <vector>
using namespace std;

vector<long long> memo; // Vector para almacenar resultados

long long fib_memo(int n) {
    if (memo[n] != -1) {
        return memo[n]; // Si ya está calculado, lo devolvemos
    }
    if (n <= 1) {
        memo[n] = n; // Caso base
    } else {
        memo[n] = fib_memo(n - 1) + fib_memo(n - 2);
    }
    return memo[n];
}

int main() {
    int numero;
    cout << "Ingrese un número: ";
    cin >> numero;

    // Inicializamos el vector con -1 (no calculado)
    memo.assign(numero + 1, -1);

    cout << "El término " << numero << " de la serie Fibonacci es: " << fib_memo(numero) << endl;

    return 0;
}
```

✅ **Lección clave:** Identificar subproblemas repetidos es el primer paso hacia la optimización. Un árbol de recursión revela instantáneamente este patrón.

Errores Comunes al Analizar Algoritmos Recursivos

✗ Error 1: Ignorar llamadas múltiples

Asumir que una función con dos llamadas recursivas tiene la misma complejidad que una con una sola. **Incorrecto:** dos llamadas generalmente duplican el trabajo en cada nivel $\rightarrow O(2^n)$ vs $O(n)$.

✗ Error 2: Confundir profundidad con total de llamadas

La **profundidad** del árbol es la memoria usada (complejidad espacial). El **total de nodos** es el tiempo. En Fibonacci: profundidad = n , pero nodos = $O(2^n)$.

✗ Error 3: Omitir la complejidad espacial

Olvidar que cada llamada activa ocupa espacio en el stack. Un algoritmo $O(n)$ en tiempo puede tener $O(n)$ en espacio por la pila de llamadas, lo que puede ser un problema crítico.

✗ Error 4: No verificar el caso base

Asumir que el algoritmo siempre termina sin verificar que el caso base es alcanzable. Sin caso base válido, la recursión es infinita y el análisis de complejidad carece de sentido.

Ejercicio Guiado: ¡Analícemos Juntos!

Apliquemos el proceso de análisis al siguiente algoritmo. Responde cada pregunta antes de ver la respuesta.

Algoritmo: Suma de Array

```
int suma(const vector<int>& arr, int n) {
    if (n == 0) { // Caso base
        return 0;
    }
    return arr[n - 1] + suma(arr, n - 1);
}

int main() {
    vector<int> arr = {1, 2, 3, 4, 5};
    int n = arr.size();

    cout << "La suma de los elementos es: " << suma(arr, n) << endl;

    return 0;
}
```



Pregunta 1: ¿Cuántas llamadas recursivas hace por nivel?



Pregunta 2: ¿En cuánto se reduce el tamaño del problema?



Pregunta 3: ¿Cuál es el trabajo en cada nodo?

Guía de Análisis

01

Llamadas: 1 sola llamada recursiva → no hay explosión exponencial

02

Reducción: $n \rightarrow n-1$ en cada llamada → profundidad = n

03

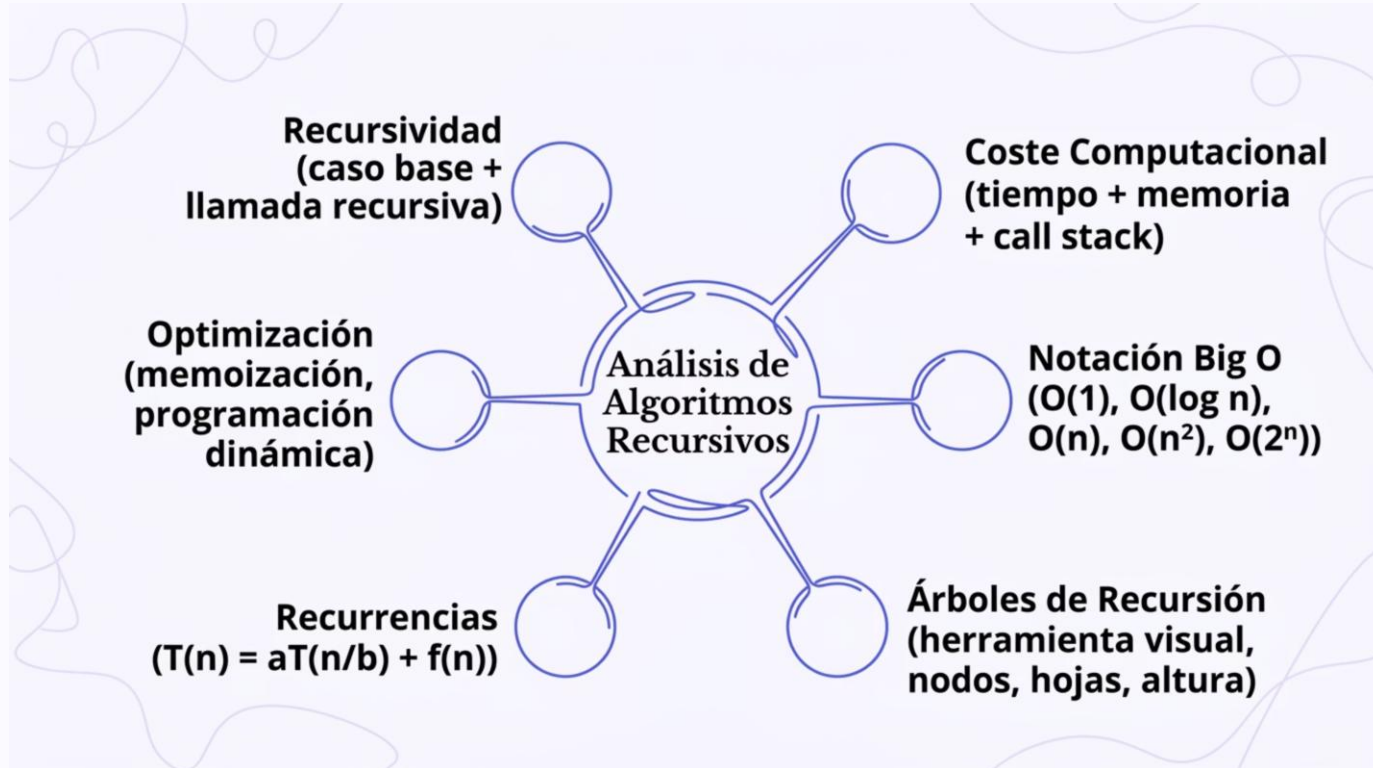
Trabajo por nodo: $O(1)$ — solo una suma y acceso a array

04

Total: $n \text{ niveles} \times O(1) = O(n)$ tiempo, $O(n)$ espacio

Resumen Visual: Mapa Conceptual

Todo lo aprendido en esta sesión, organizado en una sola vista. La complejidad de algoritmos recursivos se entiende siguiendo el camino: código → árbol → recurrencia → Big O.



Conclusiones y Recomendaciones Clave



La recursión tiene un coste real

Cada llamada recursiva consume tiempo y memoria. Nunca asumir que un algoritmo recursivo es eficiente solo por ser elegante: siempre analiza su complejidad.

$O(\log n)$

Búsqueda Binaria

El algoritmo más eficiente estudiado hoy



El árbol de recursión es tu mejor herramienta

Antes de calcular Big O, dibuja el árbol. Visualizar las llamadas te permite detectar problemas como la repetición de subproblemas o el crecimiento exponencial.

$O(2^n)$

Fibonacci Simple

El más ineficiente — evitar en producción



Optimiza cuando importa

La memoización y la programación dinámica pueden transformar un $O(2^n)$ en $O(n)$. Identifica subproblemas repetidos y almacena sus resultados.

4

Pasos del Análisis

Identificar → Reducción → Trabajo → Sumar

Glosario de Términos Clave

Referencia rápida con los conceptos fundamentales cubiertos en esta sesión para consulta durante el estudio.

Término	Definición	Ejemplo
Recursividad	Técnica donde una función se llama a sí misma para resolver subproblemas	factorial(n)
Caso Base	Condición que detiene la recursión y devuelve un resultado directo	if n == 0: return 1
Call Stack	Pila de ejecución que almacena marcos activos de llamadas recursivas	Profundidad = O(n)
Notación Big O	Describe el crecimiento del tiempo de ejecución en el peor caso	O(log n), O(2 ⁿ)
Reurrencia	Ecuación que expresa T(n) en función de T de tamaños menores	T(n) = T(n/2) + O(1)
Árbol de Recursión	Representación visual jerárquica de todas las llamadas recursivas	Árbol de fib(5)
Memoización	Técnica de guardar resultados ya calculados para evitar recomputación	@lru_cache en Python
Stack Overflow	Error producido cuando la pila de llamadas supera su capacidad máxima	Recursión infinita

Lecturas y Recursos Recomendados

Profundiza en los temas de esta sesión con estos recursos cuidadosamente seleccionados para el nivel introductorio-intermedio.

1

Introduction to Algorithms — CLRS

Cormen, Leiserson, Rivest & Stein. Capítulos 2, 3 y 4. El libro de referencia definitivo para análisis de algoritmos en cualquier carrera de Ingeniería.

2

Algorithms — Sedgewick & Wayne

Excelente para visualizaciones y ejemplos en código. Ideal como complemento práctico al CLRS con enfoque en implementación real.

3

Khan Academy — Algorithms

Curso gratuito online con videos interactivos sobre complejidad, recursividad y notación Big O. Perfecto para repasar antes de exámenes.

4

Visualgo.net

Plataforma interactiva para visualizar algoritmos en ejecución, incluyendo árboles de recursión animados en tiempo real. Completamente gratuita.



Meta de la próxima sesión: Resolver recurrencias usando el Teorema Maestro y analizar la complejidad del algoritmo MergeSort completo usando su árbol de recursión característico.